

What is Snowpipe?

Snowpipe is a mechanism in Snowflake used to **load high-frequency or streaming data automatically** as soon as files arrive in a stage.

It enables **near real-time or micro-batch data ingestion** without manually running COPY commands.

Snowpipe continuously monitors staged files and automatically loads them into Snowflake tables.

Example real-world streaming data:

- Banking transaction systems
- IoT device event streams
- Website clickstream data
- Application log files

These systems generate data continuously, and Snowpipe ensures the data gets loaded **within seconds or minutes**.

Snowpipe Architecture

Snowpipe works using a **serverless architecture**.

Unlike bulk loading using **COPY INTO**, Snowpipe does **not use Virtual Warehouse resources**.

Instead:

- Snowflake manages compute automatically
- Scaling happens automatically
- No warehouse management required

This makes Snowpipe ideal for **continuous ingestion pipelines**.

Snowpipe Internal Concept

A **Snowpipe** contains a **COPY command**.

That means Snowpipe is essentially a **wrapper around the COPY INTO command**.

Snowpipe definition tells Snowflake:

- Which stage contains incoming files
- Which table to load data into
- Which file format to use

Whenever new files arrive in the stage, Snowpipe executes the COPY command automatically.

High-Level Steps for Loading Data Using Snowpipe

There are **four main steps**.

Step 1 – Stage the Data

First, data must arrive in a **stage location**.

The stage could be:

- Amazon S3
- Azure Blob Storage
- Google Cloud Storage
- Internal Snowflake Stage

Example:

Streaming application sends files to **S3 bucket continuously**.

Snowpipe monitors this location.

Step 2 – Test the COPY Command

Before creating Snowpipe, we must verify that the COPY command works correctly.

Example process:

1. Create target table
2. Upload sample file to stage
3. Run COPY INTO manually
4. Verify data loads correctly

Once confirmed, the same COPY command will be used inside the Snowpipe definition.

Step 3 – Create the Pipe

A **Pipe** is the Snowflake object that defines the Snowpipe.

The pipe contains the COPY command that loads data.

Example structure:

```
CREATE PIPE my_pipe
AS
COPY INTO target_table
FROM @my_stage
FILE_FORMAT = (TYPE = CSV);
```

This tells Snowflake:

Whenever new files arrive in **@my_stage**, load them into **target_table**.

Step 4 – Configure Event Trigger

Snowpipe needs a **trigger mechanism** to detect new files.

There are **two methods**.

Method 1 – Cloud Event Notification (Recommended)

This is the **most commonly used production architecture**.

Cloud platforms provide notification services.

Examples:

AWS → SQS / SNS

Azure → Event Grid

GCP → Pub/Sub

When a new file arrives in the storage bucket:

1. Cloud generates event notification
2. Notification triggers Snowpipe
3. Snowpipe loads the file automatically

This approach is **fully automated and highly reliable**.

Method 2 – Trigger Using REST API

Another approach is triggering Snowpipe manually through **REST APIs**.

In this case:

1. A custom program monitors new files
2. Program calls Snowpipe REST API
3. Snowpipe loads the file

Snowflake also provides:

- Java APIs
- Python APIs

These simplify interacting with Snowpipe.

End-to-End Snowpipe Data Flow

External Data Source



Cloud Storage (S3 / Blob Storage)

↓
External Stage
↓
Cloud Event Notification
↓
Snowpipe Trigger
↓
COPY Command Execution
↓
Snowflake Table

Creating Event Notifications in AWS

Steps to configure S3 event notifications.

1. Go to **S3 Bucket**
2. Open **Properties**
3. Navigate to **Event Notifications**
4. Click **Add Notification**
5. Configure the trigger
6. Choose **SQS Queue as destination**
7. Provide **Snowpipe ARN**

The ARN can be retrieved using:

SHOW PIPES;

Once configured, whenever a new file arrives in S3:

Snowpipe automatically loads the data.

Validating Data Load

After uploading files to S3, verify data load.

Example:

```
SELECT COUNT(*)  
FROM public.transactionDetails;
```

If rows increase, Snowpipe loaded the data successfully.

Snowpipe File Processing Behavior

Snowpipe maintains **file metadata** to avoid duplicate loads.

Metadata includes:

- File name
- File path
- Load status

This prevents reloading the same file.

Snowpipe File Duplication Scenarios

Scenario 1

Same file uploaded again with **different name**

Result:

Snowpipe loads the file again.

Because filename is different.

Scenario 2

File modified but **same name**

Result:

Snowpipe does NOT load the file again.

Because metadata already contains that file name.

Recommended File Size

Snowflake recommends:

100 MB – 250 MB compressed files

Why?

- Too small files increase overhead
- Too large files slow ingestion

Balanced file size improves performance.

File Loading Order

Each Snowpipe maintains a **single queue**.

Files are added to the queue as they arrive.

However:

Multiple processes read from the queue simultaneously.

Therefore:

Snowflake **does not guarantee strict file order loading**.

Older files usually load first, but not always.

Snowpipe Charges

Snowpipe pricing is based on **number of files processed**.

Pricing model:

0.06 credits per 1000 files queued

Important points:

- Charges are per file
 - Smaller files increase cost
 - Larger optimized files reduce cost
-

Snowpipe Design Limitation

One Snowpipe loads data into **only one table**.

Therefore:

Large systems usually create **multiple pipes**.

Example:

Customer table → customer_pipe

Orders table → orders_pipe

Transactions table → transaction_pipe

Snowpipe Load History

Snowflake tracks ingestion history.

Bulk COPY loads:

Stored for **64 days**

Snowpipe loads:

Stored for **14 days**

You can query load history using:

`COPY_HISTORY()`

Loading Historic Data

Sometimes we need to load old files.

Snowpipe only loads files that trigger events.

To load historic data:

Step 1

Load old files manually using COPY INTO

Step 2

Configure Snowpipe for new files

Step 3

Run

ALTER PIPE REFRESH

This loads any files staged recently but not processed.

Resuming a Stale Pipe

If a pipe is paused for **more than 14 days**, it becomes **stale**.

When resumed:

Old event notifications may be lost.

To resume a stale pipe:

```
SELECT system$pipe_force_resume(  
'mydb.myschema.pipe1',  
'staleness_check_override'  
);
```

This command forces Snowpipe to resume execution.

Key Snowpipe Takeaways

- Snowpipe enables near real-time ingestion
- Uses serverless architecture
- No virtual warehouse required
- Automatically scales compute
- Uses event notifications to trigger loading
- Prevents duplicate file loads using metadata

Snowpipe is a **core component of modern real-time data pipelines in Snowflake**.